

Stage 6 — Production Repo Layout, HTTP API, and Integration of Stages 1–5

Goal: Take everything from Stages 1–5 (agents, orchestrator, memory, guardrails, evals, observability) and organize it into a **production-grade repository** with a clean HTTP API, Docker packaging, tests, and docs.

This stage mirrors the architecture in your screenshot (e.g., `production-ai-app/`), but adapted to your Creator Campaign Orchestrator and Postgres + pgvector stack.[web:207][web:209]

By the end of Stage 6 you will have:

- A top-level repo structure that looks and feels like a serious AI product.
- A FastAPI-based HTTP service (`/api/orchestrate`) wrapping your orchestrator and guardrails.[web:207][web:209]
- All Stage 1–5 modules slotted into clear packages (app, agents, services, security, evaluation, observability, data, scripts, tests, docs).
- Dockerfiles and scripts to run the service and migrations.

No local JSON files, no ad-hoc directories — everything runs on **Postgres + pgvector**, with your existing memory and guardrail tables.[web:181][web:185]

1. How Many Stages and Where You Are

You now have **six stages** defined end-to-end:

1. **Stage 1** — Core agent loop + tools.
2. **Stage 2** — Orchestrator + multiple agents.
3. **Stage 3** — Memory: Postgres + pgvector (short-term, long-term, episodic).
4. **Stage 4** — Guardrails & policy (pre-LLM, post-LLM, tool RBAC).
5. **Stage 5** — Evals & observability (eval harness + metrics).
6. **Stage 6** — Production repo layout + HTTP API (this document).

After Stage 6, you are at a level where many teams would ship an internal beta.[web:202][web:204]

If you want to keep going, recommended **optional stages** would be:

- **Stage 7** — Auth, multi-tenant support, and rate limiting.
- **Stage 8** — Billing, quotas, and feature flags.

Those are business/product layers; architecturally, Stage 6 gives you a solid, enterprise-style backbone.

2. Comparison to the Screenshot Architecture

Your screenshot shows a `production-ai-app/` layout with folders like:

- `app/` — FastAPI entry, config, models.
- `components/` — retrievers, rerankers.
- `services/` — RAG pipeline, cache, router.
- `agents/` — graders, decomposers, routers + tools.
- `security/` — input/content/output filters.
- `evaluation/` — golden dataset, offline eval, online monitor.
- `observability/` — tracer, feedback, cost tracker.
- `data/`, `scripts/`, `frontend/`, `tests/`, `docs/`, etc.

This is **strong and aligned** with modern LLM app patterns: clear separation of concerns, evals/observability as first-class, and security as a dedicated layer.[web:207][web:209][web:212]

Stage 6 maps your Stage 1–5 modules into a similar structure and fills a few gaps (Postgres + pgvector, guardrails, eval harness) so it's coherent for your Creator Orchestrator.

3. Final Repo Layout (Creator Orchestrator)

At the top level:

```
creator-orchestrator/
├── app/
│   ├── main.py           # FastAPI app entrypoint
│   ├── config.py         # Pydantic settings (env, DB, keys)
│   ├── models.py         # Pydantic API schemas (requests/responses)
│   ├── orchestrator.py   # Orchestrator wiring wrapper
│   └── Dockerfile
├── agents/
│   ├── base.py
│   ├── research_agent.py
│   ├── writer_agent.py
│   ├── reviewer_agent.py
│   └── __init__.py
├── services/
│   ├── memory_service.py # Thin façade over memory module
│   ├── guardrail_service.py # Thin façade over guardrails
│   ├── eval_service.py   # Runs eval harness / reports
│   └── conversation.py   # Session orchestration helpers
├── components/
│   ├── retrievers/
│   │   ├── pgvector_retriever.py
│   │   └── __init__.py
```

```
|   |   | rerankers/
|   |   | |   simple_reranker.py
|   |   | |   __init__.py
|
|   | prompts/
|   | |   templates.py
|   | |   registry.py
|   | |   |   system/
|   | |   |   |   research.txt
|   | |   |   |   writer.txt
|   | |   |   |   reviewer.txt
|
|   | tools/
|   | |   registry.py
|   | |   calculator.py
|   | |   web_search.py
|   | |   video_tools.py
|   | |   __init__.py
|
|   | security/
|   | |   input_guard.py           # wraps pre-LLM guardrails
|   | |   content_filter.py      # wraps post-LLM / policy
|   | |   tool_guard.py          # wraps RBAC for tools
|   | |   __init__.py
|
|   | memory/
|   | |   db.py
|   | |   models.py
|   | |   short_term.py
|   | |   long_term.py
|   | |   episodic.py
|   | |   __init__.py
|
|   | evaluation/
|   | |   golden_dataset.json
|   | |   metrics.py
|   | |   judge.py
|   | |   harness.py
|   | |   store.py
|
|   | observability/
|   | |   tracing.py
|   | |   metrics.py
|   | |   __init__.py
|
|   | data/
|   | |   raw/
|   | |   |   processed/
|   | |   |   |   index_config/
|
|   | scripts/
|   | |   seed.py                # seed DB (roles, tool_policies, sample memories)
|   | |   migrate.py            # simple migration runner
|   | |   healthcheck.py
|
|   | tests/
```

```

|   ├── test_agents.py
|   ├── test_memory.py
|   ├── test_guardrails.py
|   ├── test_orchestrator.py
|   └── test_api.py
|
|   ├── docs/
|   │   ├── architecture.md
|   │   ├── api-reference.md
|   │   └── deployment.md
|   |
|   ├── infra/
|   │   ├── docker-compose.yml
|   │   ├── pyproject.toml
|   │   └── README.md
|   └── .env.example

```

This keeps your Stage 1–5 code but arranges it so that anyone can open the repo and immediately see:

- where the API is,
- where agents live,
- where memory/guardrails/evals/observability are.

4. Step-by-Step: Create the app/ Layer (HTTP API)

4.1 app/config.py — central settings

Use Pydantic settings to load env vars in one place, as recommended for production LLM apps. [web:207][web:209]

```

# app/config.py
from pydantic import BaseSettings, AnyUrl

class Settings(BaseSettings):
    openai_api_key: str
    orchestrator_model: str = "gpt-4o"
    agent_model: str = "gpt-4o-mini"
    database_url: AnyUrl

    # future: anthropic_api_key, gemini_api_key, feature flags

    class Config:
        env_file = ".env"

settings = Settings()

```

You then import settings everywhere you previously imported individual env vars.

4.2 app/models.py — API request/response schemas

```
# app/models.py
from typing import List, Optional
from pydantic import BaseModel

class OrchestrateRequest(BaseModel):
    topic: str
    brief: str
    user_id: Optional[str] = None

class Artifact(BaseModel):
    kind: str # 'clip', 'post', 'email'
    content: str

class OrchestrateResponse(BaseModel):
    run_id: str
    session_id: str
    artifacts: List[Artifact]
    metrics: dict
```

These types make your HTTP API self-documenting and are aligned with FastAPI auto-docs. [web:207]

4.3 app/orchestrator.py — wiring function

This file adapts your Stage 2 orchestrator to the API layer.

```
# app/orchestrator.py
from agents.research_agent import ResearchAgent
from agents.writer_agent import WriterAgent
from agents.reviewer_agent import ReviewerAgent
from orchestrator.code_driven import CodeDrivenOrchestrator

def build_orchestrator() -> CodeDrivenOrchestrator:
    research_agent = ResearchAgent()
    writer_agent = WriterAgent()
    reviewer_agent = ReviewerAgent()
    return CodeDrivenOrchestrator(research_agent, writer_agent, reviewer_agent)
```

Later you can add dependency injection, but for Stage 6 this is enough.

4.4 app/main.py — FastAPI endpoint

```
# app/main.py
from fastapi import FastAPI, Depends, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from uuid import uuid4

from app.config import settings
from app.models import OrchestrateRequest, OrchestrateResponse, Artifact
from app.orchestrator import build_orchestrator
from memory import short_term, episodic
from observability.tracing import run_timer

app = FastAPI(title="Creator Orchestrator API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # tighten later
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

def get_orchestrator():
    return build_orchestrator()

@app.get("/health")
async def health():
    return {"status": "ok"}

@app.post("/api/orchestrate", response_model=OrchestrateResponse)
async def orchestrate(payload: OrchestrateRequest,
                      orchestrator = Depends(get_orchestrator)):
    # 1) Create session + run
    session_id = short_term.create_session(user_id=payload.user_id, topic=payload.topic)
    run_id = episodic.start_run(session_id, orchestrator="http-api", metadata={})

    artifacts = []

    try:
        with run_timer(run_id, session_id) as metrics:
            result = orchestrator.run(
                topic=payload.topic,
                user_id=payload.user_id,
                session_id=session_id,
                run_id=run_id,
                brief=payload.brief,
            )

        episodic.finish_run(run_id, status="success")

        # Map orchestrator result to Artifact list
        for post in result.get("posts") or []:
```

```

        artifacts.append(Artifact(kind="post", content=post))
    if result.get("email"):
        artifacts.append(Artifact(kind="email", content=result["email"]))
    for clip in result.get("clips") or []:
        artifacts.append(Artifact(kind="clip", content=clip))

    return OrchestrateResponse(
        run_id=run_id,
        session_id=session_id,
        artifacts=artifacts,
        metrics=result.get("metrics", {}),
    )

except Exception as e:
    episodic.finish_run(run_id, status="error", error=str(e))
    raise HTTPException(status_code=500, detail="Internal error")

```

This endpoint integrates:

- Stage 2 orchestrator.
- Stage 3 memory (`session_id`, `run_id`).
- Stage 5 observability (`run_timer`).

Guardrails and tool RBAC still operate inside agents and tools as previously wired.

5. Mapping Stages 1–5 Into the New Layout

5.1 Stage 1 — Agents & Tools

- `agents/base.py`, `agents/*_agent.py` remain as before.
- `tools/registry.py` and other tools stay but now live under `tools/` at the root.
- They are imported from `app/main.py` only through the orchestrator.

This matches the screenshot's `agents/` and `tools/` split, where tools are pluggable modules underneath agents.[\[web:207\]](#)

5.2 Stage 2 — Orchestrator & Prompts

- Orchestrator implementation stays in a dedicated package (e.g. `orchestrator/code_driven.py`).
- Prompts and templates are moved into `prompts/`:
 - `prompts/templates.py` contains helper functions.
 - `prompts/system/*.txt` store system prompts as static files (easy to version and localize).

This aligns with the screenshot's `prompts/templates.py` & `prompts/registry.py` pattern for versioned prompts.[\[web:207\]](#)

5.3 Stage 3 — Memory (Postgres + pgvector)

All memory code moves into `memory/`:

- `memory/db.py` — connection helper (psycopg).
- `memory/models.py` — dataclasses for `Session`, `Message`, `MemoryRecord`, `Run`, `Event`.
- `memory/short_term.py`, `long_term.py`, `episodic.py` — as before.

`app/main.py`, `agents/base.py`, and `evals/harness.py` all import from `memory/` instead of directly talking to Postgres. This keeps a clean data access layer, as recommended when using pgvector inside Postgres.[web:181][web:186]

5.4 Stage 4 — Guardrails & Security

Your Stage 4 modules map naturally to a `security/` package:

- `security/input_guard.py` wraps `guardrails.service.apply_pre_llm_guard`.
- `security/content_filter.py` wraps `guardrails.service.apply_post_llm_guard`.
- `security/tool_guard.py` wraps `guardrails.service.check_tool_guard`.

This matches the screenshot's separate `security/` folder and clearly signals where safety policies live.[web:189][web:192]

5.5 Stage 5 — Evaluation & Observability

- All eval-related code moves under `evaluation/`:
 - `evaluation/metrics.py`, `evaluation/judge.py`, `evaluation/harness.py`, `evaluation/store.py`.
 - `evaluation/golden_dataset.json` is your golden set of topics & briefs.
- Observability modules move under `observability/`:
 - `observability/tracing.py`, `observability/metrics.py`.

This matches the screenshot's `evaluation/` and `observability/` folders and reflects modern patterns for separate eval and monitoring layers.[web:202][web:204][web:207][web:212]

6. Scripts, Tests, Docs, and Infra

6.1 Scripts

Under `scripts/`:

- `seed.py` — insert initial roles, tool_policies, guardrail_policies, and sample memories.
- `migrate.py` — simple migration runner that applies SQL files from `migrations/`.
- `healthcheck.py` — small script to ping `/health` and exit with non-zero on failure (for Docker/Kubernetes liveness probes).[web:207]

6.2 Tests

Under tests/:

- `test_agents.py` — unit tests for individual agents (mocking LLM calls).
- `test_memory.py` — tests for session/message/long-term memory.
- `test_guardrails.py` — tests for pre/post/tool guardrails (e.g., injection, PII).[web:190][web:195]
- `test_orchestrator.py` — ensures orchestrator returns structured output.
- `test_api.py` — end-to-end test with FastAPI TestClient.

This mirrors the screenshot's tests/test_retrieval.py, etc., but adjusted to your feature set.

6.3 Docs

Under docs/:

- `architecture.md` — high-level architecture, including the stages and directory layout.
- `api-reference.md` — HTTP endpoints, request/response schemas.
- `deployment.md` — how to run locally, in Docker, and in the cloud.

Good architecture docs are part of enterprise practice for AI systems, especially when multiple teams work on the stack.[web:202][web:204]

6.4 Infra & Docker

Under infra/:

- `docker-compose.yml` — multi-container setup:
 - `api` (FastAPI service).
 - `db` (Postgres + pgvector image, with migrations volume).
 - optional: `otel-collector` or `prometheus` in later stages.[web:207][web:214]
- `pyproject.toml` — Python package + dependencies.
- `README.md` — quickstart that references Stages 1–6.

This is where you later add Kubernetes manifests or Terraform, but Stage 6 focuses on Docker + docker-compose.

7. Step-by-Step Migration from Stages 1–5 to Stage 6 Layout

1. Create new repo root `creator-orchestrator/`.
2. Copy existing Stage 1–5 modules into their target folders (`agents/`, `tools/`, `memory/`, `guardrails/`, `evaluation/`, `observability/`).
3. Create `app/` with `config.py`, `models.py`, `orchestrator.py`, and `main.py` as shown.
4. Update imports:

- Replace direct env access (`os.getenv`) with `from app.config import settings`.
- Replace old module paths (`from orchestrator.code_driven`) with the new package paths.

5. Add `infra/docker-compose.yml` wiring api and db together.

6. Add tests and docs incrementally; start with `test_api.py` hitting `/health` and `/api/orchestrate`.

After this, you can run:

```
# inside creator-orchestrator/
uvicorn app.main:app --reload
```

and your entire Stage 1–5 functionality is accessible via HTTP.

8. Why This Architecture Is Solid (and What’s Left)

Why it’s correct and enterprise-aligned

- **Clear separation of concerns:** API, agents, memory, security, evals, and observability are split into explicit packages, just like the strong reference architecture in your screenshot. [\[web:207\]](#)[\[web:209\]](#)
- **Single source of truth for data:** Postgres + pgvector handles sessions, memory, guardrails, evals, and metrics — consistent with current guidance that you often do not need a separate vector DB until you hit serious scale. [\[web:181\]](#)[\[web:185\]](#)[\[web:186\]](#)
- **Guardrails and evals are first-class citizens:** They live beside the core app, not as one-off scripts, which matches modern best practices. [\[web:77\]](#)[\[web:188\]](#)[\[web:202\]](#)[\[web:204\]](#)
- **Observability & API instrumentation:** Traces and metrics are part of the core design, not an afterthought. [\[web:204\]](#)[\[web:207\]](#)[\[web:209\]](#)[\[web:212\]](#)

What’s left (future stages, optional)

- **Stage 7 — Auth, multi-tenant support, rate limiting** (e.g., JWT, API keys, per-tenant DB tenancy). [\[web:192\]](#)[\[web:197\]](#)
- **Stage 8 — Billing, quotas, and feature flags** (Stripe/ Paddle integration, per-plan limits).

These are product features rather than core architecture. The six stages you have now form a cohesive, enterprise-style foundation you can deploy, monitor, and iterate.